

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
MINGGU 2 PERTEMUAN 2
SEMESTER III



BKPM ACARA 4

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Seiring bertambahnya jumlah halaman pada aplikasi web, penulisan HTML secara berulang pada setiap file menjadi tidak efisien. Elemen yang sama seperti header, navigasi, dan footer sebaiknya dipisah menjadi berkas tersendiri (partial) dan disatukan kembali melalui layout, sehingga perubahan cukup dilakukan pada satu tempat. PHP menyediakan mekanisme include dan require untuk menyisipkan file lain ke dalam file yang sedang dieksekusi.

Selain pemisahan tampilan, praktikum ini juga memperkenalkan dasar pemrograman berorientasi objek (OOP) di PHP, yaitu penggunaan class dan object. Dengan OOP, data dan perilaku terkait dapat dikelompokkan dalam satu kesatuan yang disebut class, sehingga kode menjadi lebih terstruktur, mudah dipelihara, dan dapat digunakan kembali.

2. Tujuan

1. Menjelaskan perbedaan include, require, include_once, dan require_once.
2. Membuat layout dengan partial (header, navbar, footer) menggunakan include.
3. Membuat class Mahasiswa dengan property private dan method getter/setter (encapsulation).
4. Menampilkan data dari beberapa object Mahasiswa ke dalam View berbentuk tabel Bootstrap.

3. Dasar Teori Singkat

Include dan Require. include dan require digunakan untuk menyisipkan isi file lain ke dalam file saat ini. Perbedaannya terletak pada penanganan kesalahan: `include` hanya memunculkan warning dan script tetap berjalan apabila file tidak ditemukan, sedangkan `require` memunculkan fatal error dan menghentikan script. Varian `include_once` dan `require_once` memastikan file hanya dimasukkan satu kali untuk mencegah redeclaration. Praktik terbaiknya, `require_once` digunakan untuk file penting seperti konfigurasi dan class, sedangkan `include` digunakan untuk partial view.

Layout dan Partial. Layout adalah kerangka halaman (header, konten, footer) yang dipakai berulang oleh banyak halaman, sedangkan partial adalah potongan kecil tampilan (navbar, footer) yang disisipkan ke dalam layout maupun view lain. Pada praktikum ini, layout utama

(`main.php`) memuat navbar dan footer melalui include, serta memuat konten halaman melalui variabel `$content` yang di-require.

Class dan Object. Class adalah blueprint atau cetakan, sedangkan object adalah instance konkret dari class tersebut. Pada praktikum ini dibuat class `Mahasiswa` dengan property `private (nim, nama, prodi)` yang hanya dapat diakses melalui method getter dan setter. Konsep ini disebut encapsulation, yaitu menyembunyikan detail internal dan hanya mengekspos apa yang diperlukan, sehingga data tidak dapat diubah secara sembarangan dari luar class.

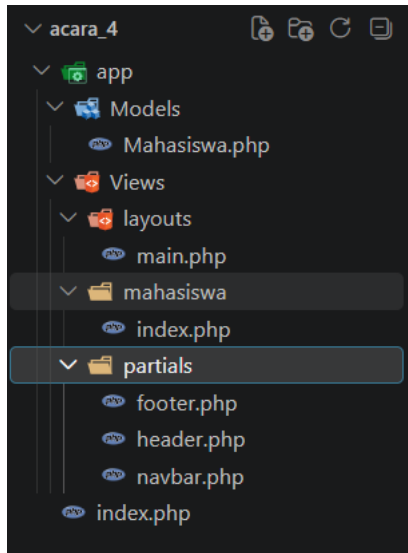
4. Alat dan Bahan

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon
- PHP
- HTML, CSS, Bootstrap
- Modul Praktikum (BKPM Acara 4)

HASIL PRAKTIK

1. Struktur Folder Project

Mengikuti langkah kerja pada BKPM, dibuat struktur folder berikut di dalam project (di dalam htdocs untuk XAMPP):



2. Membuat Partial (header, navbar, footer)

Ketiga partial berikut dibuat agar header, navigasi, dan footer dapat digunakan berulang tanpa duplikasi kode.

```
app > Views > partials > header.php
1  <!-- app/Views/partial/header.php -->
2  <head>
3      <meta charset="UTF-8">
4      <meta name="viewport" content="width=device-width, initial-scale=1">
5      <title>Data Mahasiswa</title>
6      <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="styles
7  </head>
```

```
app > Views > partials > navbar.php
1  <!-- app/Views/partial/navbar.php -->
2  <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
3      <div class="container">
4          <a class="navbar-brand" href="#">Workshop Web Server</a>
5          <div class="collapse navbar-collapse">
6              <ul class="navbar-nav">
7                  <li class="nav-item"><a class="nav-link active" href="#">Data Mahasiswa</a></li>
8              </ul>
9          </div>
10     </div>
11 </nav>
```

```
app > Views > partials > footer.php
1 <!-- app/Views/partials/footer.php -->
2 <footer class="bg-light text-center py-3 mt-5 border-top">
3 |   <p class="mb-0">&copy; <?php echo date("Y"); ?> Workshop Sistem Informasi Web Server - TIF330805
4 </footer>
```

3. Membuat Layout Utama

Layout utama menyatukan header, navbar, konten halaman, dan footer menggunakan include dan require.

```
app > Views > layouts > main.php
1 <!-- app/Views/layouts/main.php -->
2 <!DOCTYPE html>
3 <html lang="id">
4 <?php include __DIR__ . '/../partials/header.php'; ?>
5 <body>
6 |   <?php include __DIR__ . '/../partials/navbar.php'; ?>
7 |   <main class="container mt-4">
8 | |   <?php require $content; ?>
9 |   </main>
10 |   <?php include __DIR__ . '/../partials/footer.php'; ?>
11 </body>
12 </html>
```

4. Membuat Class Mahasiswa (Model)

Class Mahasiswa dibuat dengan property private serta method getter untuk menerapkan encapsulation. Ditambahkan pula method getAngkatan() untuk Tugas Mandiri, yang mengambil angkatan dari dua digit awal NIM.

```
1  <?php
2  // app/Models/Mahasiswa.php
3  namespace App\Models;
4
5  class Mahasiswa
6  {
7      private string $nim;
8      private string $nama;
9      private string $prodi;
10
11     public function __construct(string $nim, string $nama, string $prodi)
12     {
13         $this->nim = $nim;
14         $this->nama = $nama;
15         $this->prodi = $prodi;
16     }
17
18     public function getNim(): string
19     {
20         return $this->nim;
21     }
22
23     public function getNama(): string
24     {
25         return $this->nama;
26     }
27
28     public function getProdi(): string
29     {
30         return $this->prodi;
31     }
32
33     public function setNama(string $nama): void
34     {
35         if (strlen($nama) < 3) {
36             throw new \InvalidArgumentException("Nama terlalu pendek");
37         }
38         $this->nama = $nama;
39     }
40
41     // Tugas Mandiri: menentukan angkatan dari 2 digit awal NIM
42     // Format NIM diasumsikan: TTNNNNN, TT = 2 digit tahun masuk (mis. 24 -> 2024)
43     public function getAngkatan(): string
44     {
45         $dua_digit = substr($this->nim, 0, 2);
46         if (!ctype_digit($dua_digit)) {
47             return "Tidak diketahui";
48         }
49         return "20" . $dua_digit;
50     }
51
52     public function getLabel(): string
53     {
54         return "{$this->nim} - {$this->nama} ({$this->prodi})";
55     }
56 }
57
```

5. Entry Point dan Pengiriman Data ke View

File index.php membuat beberapa object Mahasiswa lalu mengirimkannya ke View melalui variabel \$daftarMahasiswa, kemudian me-require layout utama.

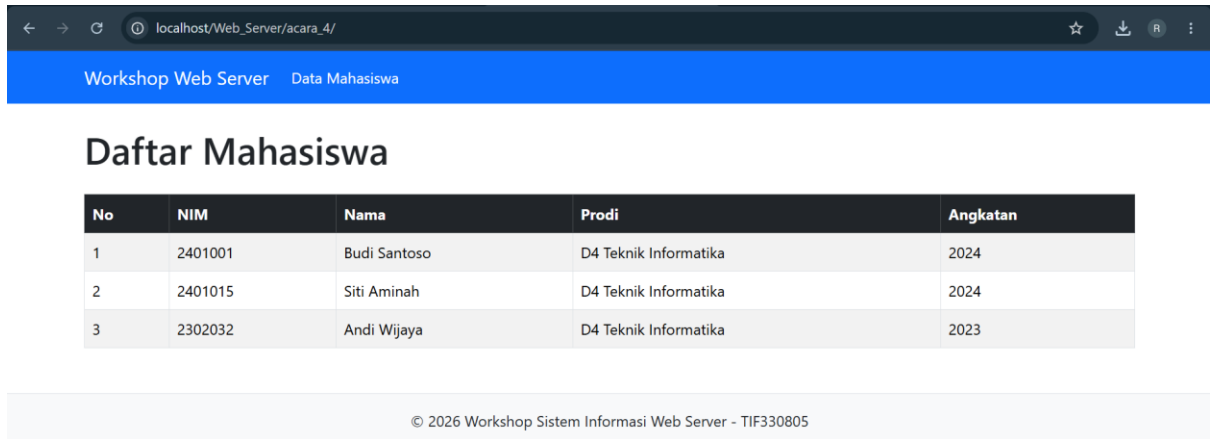
```
index.php
1  <?php
2  // index.php
3  require_once __DIR__ . '/app/Models/Mahasiswa.php';
4
5  use App\Models\Mahasiswa;
6
7  // Membuat beberapa object Mahasiswa
8  $daftarMahasiswa = [
9      new Mahasiswa("2401001", "Budi Santoso", "D4 Teknik Informatika"),
10     new Mahasiswa("2401015", "Siti Aminah", "D4 Teknik Informatika"),
11     new Mahasiswa("2302032", "Andi Wijaya", "D4 Teknik Informatika"),
12 ];
13
14 $content = __DIR__ . '/app/Views/mahasiswa/index.php';
15 require __DIR__ . '/app/Views/layouts/main.php';
```

Isi View mahasiswa/index.php menampilkan data setiap object dalam tabel Bootstrap, memanggil method getter (bukan property secara langsung), sesuai prinsip encapsulation.

```
app > Views > mahasiswa > index.php
1  <!-- app/Views/mahasiswa/index.php -->
2  <h1 class="mb-4">Daftar Mahasiswa</h1>
3  <table class="table table-bordered table-striped">
4      <thead class="table-dark">
5          <tr>
6              <th>No</th>
7              <th>NIM</th>
8              <th>Nama</th>
9              <th>Prodi</th>
10             <th>Angkatan</th>
11         </tr>
12     </thead>
13     <tbody>
14         <?php foreach ($daftarMahasiswa as $i => $mhs): ?>
15             <tr>
16                 <td><?php echo $i + 1; ?></td>
17                 <td><?php echo htmlspecialchars($mhs->getNim()); ?></td>
18                 <td><?php echo htmlspecialchars($mhs->getNama()); ?></td>
19                 <td><?php echo htmlspecialchars($mhs->getProdi()); ?></td>
20                 <td><?php echo htmlspecialchars($mhs->getAngkatan()); ?></td>
21             </tr>
22         <?php endforeach; ?>
23     </tbody>
24 </table>
```

6. Hasil Tampilan

Halaman diakses melalui `http://localhost/belajar-oop/index.php` dan menghasilkan tampilan berikut.



The screenshot shows a web browser window with the address bar displaying `localhost/Web_Server/acara_4/`. The page has a blue header with the text "Workshop Web Server" and "Data Mahasiswa". Below the header, the title "Daftar Mahasiswa" is centered. A table with 5 columns (No, NIM, Nama, Prodi, Angkatan) contains 3 rows of student data. At the bottom, a footer displays the copyright notice: "© 2026 Workshop Sistem Informasi Web Server - TIF330805".

No	NIM	Nama	Prodi	Angkatan
1	2401001	Budi Santoso	D4 Teknik Informatika	2024
2	2401015	Siti Aminah	D4 Teknik Informatika	2024
3	2302032	Andi Wijaya	D4 Teknik Informatika	2023

© 2026 Workshop Sistem Informasi Web Server - TIF330805

Kolom Angkatan pada tabel terisi otomatis oleh method `getAngkatan()`, misalnya NIM 2401001 dan 2401015 menghasilkan angkatan 2024, sedangkan NIM 2302032 menghasilkan angkatan 2023, sesuai dua digit awal masing-masing NIM.

7. Analisis dan Pembahasan

Pemisahan header, navbar, dan footer ke dalam partial membuat perubahan tampilan cukup dilakukan pada satu file, tanpa perlu menyunting setiap halaman satu per satu. Layout utama hanya bertugas menyatukan potongan-potongan tersebut melalui `include`, sedangkan konten spesifik setiap halaman dikirim lewat variabel `$content` yang di `require`, sehingga layout dapat dipakai ulang oleh halaman lain di luar Mahasiswa.

Pada sisi OOP, property `nim`, `nama`, dan `prodi` dideklarasikan `private` sehingga tidak dapat diakses langsung dari luar class, misalnya melalui `$mhs->nama`. Data hanya dapat dibaca melalui method getter seperti `getNama()`, dan diubah melalui `setNama()` yang sudah dilengkapi validasi panjang nama minimal 3 karakter. Pendekatan ini adalah penerapan `encapsulation` yang mencegah data diubah secara sembarangan dan memudahkan penambahan validasi di kemudian hari.

Method `getAngkatan()` pada Tugas Mandiri memanfaatkan `substr()` untuk mengambil dua digit awal NIM, kemudian menambahkan awalan "20" agar menjadi tahun angkatan empat digit. Validasi `ctype_digit()` ditambahkan agar method tetap aman apabila format NIM tidak sesuai asumsi.

KESIMPULAN

1. Include dan require dapat digunakan untuk memecah tampilan menjadi partial (header, navbar, footer) sehingga kode HTML tidak perlu diulang pada setiap halaman; require_once cocok untuk file penting seperti class agar tidak dideklarasikan dua kali.
2. Layout utama menyatukan partial dan konten halaman melalui variabel \$content, sehingga struktur tampilan aplikasi menjadi konsisten dan mudah dipelihara.
3. Class Mahasiswa menerapkan encapsulation dengan property private dan method getter/setter, sehingga data hanya dapat diakses dan diubah melalui cara yang telah ditentukan oleh class.
4. Beberapa object Mahasiswa dapat dikirim ke View dan ditampilkan dalam tabel Bootstrap dengan memanggil method getter masing-masing object di dalam perulangan foreach.
5. Method tambahan seperti getAngkatan() menunjukkan bagaimana logika pengolahan data dapat diletakkan di dalam class, sehingga View cukup memanggil method tanpa perlu mengetahui detail perhitungannya.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.